

PROCURESIGHT

**AI BASED PROCUREMENT SPEND ANALYTICS AND
COST MANAGEMENT SYSTEM**

PHASE 1 REPORT

Submitted by

AKSHAY VENKAT KRISHNA - 230701022

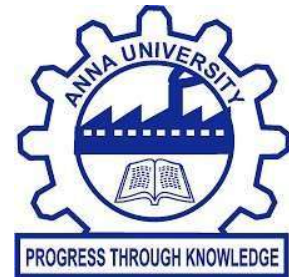
ANIRUDH S - 230701029

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled “**PROCURESIGHT – AI BASED PROCUREMENT SPEND ANALYTICS AND COST MANAGEMENT SYSTEM**” is the Bonafide work of AKSHAY VENKAT KRISHNA (230701022) and ANIRUDH S (230701029) who carried out the project work under my supervision. Certified further that, to the best of our knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates

SIGNATURE

Dr. J. Manoranjini,
Professor & Head of the Department,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai - 602105

SIGNATURE

Ms. S Kaviya,
Supervisor
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai - 602105

Submitted to Project Viva Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

ProcureSight is a Phase 1 local procurement decision-support implementation that converts validated purchase-order data into auditable spend analytics, internal price benchmarks, prioritized review opportunities, and optional grounded advisory narratives. The implemented workflow accepts a defined CSV structure through command-line ingestion, validates and normalizes the records into PostgreSQL, computes SQL-first analytics, derives supplier-level weighted prices and item-level P25 benchmarks, and detects deterministic price optimization, contract leakage, supplier consolidation, and supplier performance opportunities. The user interface is implemented in React and TypeScript, while the backend uses FastAPI, SQLAlchemy, Alembic, and PostgreSQL running locally through Docker Compose. The advisory layer uses the OpenAI Responses API with a configurable gpt-5.4-mini default and deterministic fallback; financial values are computed in the backend before any narrative is generated. On the supplied 47,128-record dataset, the workflow processed 106 suppliers, 10 countries, 10 categories, 50 items, and 5 business units with total spend of USD 411,183,335.24. The isolated full-engine execution produced 50 internal benchmarks and 254 active opportunities. The USD 12,512,042.6726 result is strictly a price-optimization estimated potential derived from internal comparison rules; it is not a realized saving or an external-market benchmark. Dataset provenance is not verified from the repository, so the dataset is not attributed to a company or external source. The implementation demonstrates reproducible candidate generation and explainable decision support while retaining human review for action and savings realization.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution. Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. J. Manoranjini, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, Ms. S Kaviya, Department of Computer Science and Engineering for her valuable guidance throughout the course of the project. We thank our Project Coordinator, Mr. Madhusudhanan, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

AKSHAY VENKAT KRISHNA: 230701022

ANIRUDH S: 230701029

TABLE OF CONTENTS

Section	Page
Abstract	iii
List of Tables	iv
List of Figures	v
List of Abbreviations	vi
Chapter 1 Introduction	9
Chapter 2 Literature Context and Cautious Research Gap	12
Chapter 3 System Analysis	20
Chapter 4 System Design	23
Chapter 5 Methodology	27
Chapter 6 Implementation	34
Chapter 7 Results and Discussion	38
Chapter 8 Conclusion and Future Work	41
References	43
Appendices	44 - 45

LIST OF TABLES

Number	Caption
Table 1.1	Objectives and evidence
Table 2.1	Comparison of Literature and ProcureSight
Table 3.1	Functional and non-functional requirements
Table 4.1	Technology stack
Table 4.2	Database schema
Table 5.1	Dataset profile
Table 5.2	Formula catalogue
Table 5.3	Opportunity rules and thresholds
Table 6.1	API inventory
Table 6.2	Testing summary
Table 7.1	Verified results
Table 7.2	Opportunity results
Table 8.1	Limitations and future work

LIST OF FIGURES

Figure No.	Figure Title
Figure 4.1	Architecture of ProcureSight
Figure 4.2	CSV Ingestion and Analytics Data Flow
Figure 4.3	Normalized Entity Relationship Structure
Figure 5.1	Internal P25 Price Benchmarking Flow
Figure 5.2	Opportunity Lifecycle
Figure 6.1	Live Dashboard Screenshot
Figure 6.2	AI Advisor Screenshot

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
AOV	Average Order Value
CSV	Comma Separated Values
ER	Entity Relationship
FX	Foreign Exchange
KPI	Key Performance Indicator
LLM	Large Language Model
ORM	Object Relational Mapper
P25	25th Percentile
PO	Purchase Order
RBAC	Role Based Access Control
USD	United States Dollar

CHAPTER 1

INTRODUCTION

1.1 General

Procurement operations create purchase-order records that combine supplier, item, quantity, price, contract, delivery, and quality information. When this information is assessed only as raw transactions or isolated summaries, a reviewer may have difficulty tracing a potential issue to its supporting data. ProcureSight is implemented as a local decision-support workflow intended to transform a defined purchase-order CSV into reviewable and explainable procurement findings.

1.2 Background

The implemented system separates calculation from narrative generation. Deterministic SQL and Python services calculate KPIs, price benchmarks, confidence, severity, and priority. The optional advisory layer receives only verified opportunity context and produces a constrained narrative or a deterministic fallback. This separation keeps financial values attributable to backend calculations rather than to generative text.

1.3 Problem Statement

The project addresses the need for an auditable path from purchase-order data to potential procurement review candidates. The implemented problem scope includes price variance against internal item benchmarks, material off-contract spend, supplier-consolidation review, and low supplier-performance scores. A finding is not a procurement decision and is not a guaranteed or realized saving.

1.4 Existing System Context

This report does not assert a universal characterization of commercial procurement platforms because that would require verified comparative sources. Within the verified project scope, a CSV-to-dashboard workflow is required to validate data, expose analytics, establish internal comparisons, and retain the calculation context behind each finding. Authentication, scheduled refresh, external market benchmarking, realized-savings tracking, and production deployment are outside the current implementation.

1.5 Proposed System

ProcureSight implements command-line CSV ingestion, Pydantic validation, normalized relational storage, SQL-first analytics, internal P25 price benchmarking, deterministic opportunity detection, priority ranking, FastAPI endpoints, a React dashboard, and an optional constrained LLM narrative layer. It runs in a Phase 1 local environment with PostgreSQL 16 in Docker Compose and frontend/backend development services run locally.

Table 1.1. Objectives and evidence

Objective	Verified implementation evidence
Validate defined purchase-order data	Pydantic CSV coercion, strict Boolean parsing, rejection behavior
Provide procurement KPIs	Spend, concentration, contract, quality, delivery and filtered analytics
Establish internal benchmarks	Supplier-level weighted prices and continuous P25 per item
Generate review candidates	Four deterministic opportunity types with thresholds and priority

Objective	Verified implementation evidence
Provide explainable advice	Optional constrained OpenAI Responses API narrative with deterministic fallback

1.6 Objectives

1. Validate a defined purchase-order CSV structure before persistence.
2. Normalize suppliers, categories, items, and business units in PostgreSQL.
3. Calculate spend, contract, quality, delivery, concentration, and supplier analytics.
4. Create item-level internal P25 benchmarks from supplier-level weighted prices.
5. Generate and rank deterministic review opportunities without representing them as realized savings.
6. Expose current procurement analytics through a web dashboard and REST API.

1.7 Scope

The implemented scope supports one imported currency with USD as the default presentation currency, batch CLI ingestion, explicit refresh scripts, and local PostgreSQL. It excludes FX conversion, external benchmark sources, forecasting, anomaly detection, scheduling, production operations, authentication and authorization, realized-savings workflows, rate limiting, backups, and observability.

CHAPTER 2

LITERATURE CONTEXT AND RESEARCH GAP

2.1 Literature Context

The literature survey examines recent research related to supplier selection, supplier evaluation, supply-chain risk assessment, sustainable procurement, and data-driven decision support. The reviewed studies demonstrate the use of multi-criteria decision-making, risk assessment, machine learning, and optimization techniques to support procurement and supply-chain decisions. These studies provide useful perspectives on supplier performance, transparency, risk prioritization, sustainability, and predictive analysis. The review also helps identify the limitations of existing approaches and motivates the development of ProcureSight, which focuses on integrating transactional procurement data, deterministic spend analytics, internal price benchmarking, opportunity identification, and grounded decision support within a single traceable workflow.

2.1.1 Haffar and Özceylan — Supplier Selection

Haffar and Özceylan [1] proposed a blockchain-based system for supplier selection in sustainable and leagile supply chains. The study focuses on improving transparency and traceability in supplier-selection decisions by combining blockchain technology with a reverse-auction mechanism. The research considers multiple supplier-selection criteria rather than evaluating suppliers based only on price. The study demonstrates how technology-supported supplier evaluation can improve the traceability of decision evidence.

The work is relevant to ProcureSight because procurement decisions require transparent and reviewable evidence. Based on this principle,

ProcureSight maintains the underlying calculation context for supplier performance, price comparisons and procurement opportunities.

However, unlike the blockchain-based approach, ProcureSight focuses on analytical spend management using normalized purchase-order data and deterministic calculations.

2.1.2 Runtuk et al. — Multi-Criteria Supplier Ranking

Runtuk et al. [2] presented a comparative study of Fuzzy VIKOR and Fuzzy MOORA for performance-based supplier selection. The study evaluates suppliers using multiple criteria and examines the effect of criteria weighting and sensitivity on supplier ranking. The comparison demonstrates the importance of transparent multi-criteria evaluation when supplier performance cannot be represented by a single measure.

This work is relevant to ProcureSight because supplier performance in procurement depends on several operational factors. ProcureSight therefore uses deterministic supplier-performance scoring based on price, quality, delivery and contract compliance. The scoring weights are explicitly defined, allowing the resulting supplier assessment to remain interpretable and reproducible.

2.1.3 Qiu et al. — Supply-Chain Risk Assessment

Qiu et al. [3] proposed a strategic decision-support approach for supply-chain risk management using probabilistic linguistic term sets and an extended CRADIS method. The approach addresses decision-making situations in which information may contain uncertainty or subjective judgments. The study demonstrates how multi-criteria risk assessment can be used to prioritize supply-chain risks in a structured and explainable manner.

The study is relevant to ProcureSight because procurement decisions can also involve uncertainty related to supplier performance, quality and delivery. The current ProcureSight implementation, however, does not perform probabilistic or predictive risk modeling. Instead, it uses verified operational indicators such as quality rejection rate, late-delivery rate and contract compliance. More advanced risk models are considered future work.

2.1.4 Moretti et al. — Sustainable Supplier Selection

Moretti et al. [4] introduced a Least-to-Most Range weighting method for green supplier selection. The study addresses the problem of assigning consistent importance to multiple supplier-selection criteria and demonstrates the use of structured weighting for sustainable supplier evaluation. The research emphasizes that supplier selection becomes more reliable when the relative importance of different criteria is explicitly represented.

This principle is relevant to ProcureSight because the system combines several procurement indicators when evaluating supplier performance and prioritizing opportunities. ProcureSight uses configurable deterministic weights and thresholds rather than the specific weighting method proposed by Moretti et al. This allows the project to maintain a transparent and reproducible scoring process.

2.1.5 Shahsavary et al. — Machine-Learning-Based Supply-Chain Risk

Shahsavary et al. [5] investigated risk prediction in distributed supply-chain design using machine learning and bi-objective optimization. The study combines Random Forest-based risk prediction with optimization to balance risk and cost in supply-chain decisions. The work demonstrates

how predictive models can provide inputs to subsequent optimization and sourcing decisions.

The study is relevant to the future development of ProcureSight because predictive risk analysis could extend the current deterministic procurement analytics workflow. However, predictive machine-learning models are not part of the current Phase 1 implementation. ProcureSight currently focuses on verified historical procurement metrics and deterministic opportunity detection rather than claiming predictive risk capability.

2.1.6 Research Gap

The literature reviewed in the previous sections demonstrates the importance of transparent supplier evaluation, multi-criteria decision-making, risk assessment, consistent criteria weighting, and optimization in procurement and supply-chain management. However, the reviewed studies primarily address specific decision-support objectives such as supplier selection, risk prioritization, sustainable supplier evaluation, or predictive optimization.

A practical procurement environment requires these analytical activities to be connected with transactional procurement data and operational spend information. There is therefore a need for an integrated workflow that can transform purchase-order data into measurable procurement insights, identify potential cost and compliance issues, prioritize the findings, and provide interpretable recommendations.

Another important observation from the literature is the need for transparent and traceable decision-making. Supplier rankings, risk assessments, and optimization results should be supported by explicit criteria and evidence so that procurement personnel can review and validate the results before taking action.

Based on these observations, ProcureSight addresses an implementation-level gap by integrating validated purchase-order data, normalized relational storage, deterministic spend analytics, internal price benchmarking, rule-based opportunity detection, priority scoring, and an optional grounded AI advisory layer within a single workflow. The system maintains traceability between the calculated procurement metrics and the resulting opportunities.

The project does not claim to introduce a novel AI or machine-learning algorithm. Instead, its contribution lies in integrating deterministic procurement analytics with explainable opportunity prioritization and constrained AI-based explanation while keeping the final procurement decision under human review.

2.2 Comparison and Discussion

The reviewed studies and the proposed ProcureSight system address different aspects of procurement and supply-chain decision support. Haffar and Özceylan [1] focus on transparency and traceability in supplier selection, while Runtuk et al. [2] emphasize multi-criteria supplier performance evaluation. Qiu et al. [3] address supply-chain risk prioritization under uncertainty, and Moretti et al. [4] focus on consistent weighting of criteria for sustainable supplier selection. Shahsavary et al. [5] demonstrate the use of machine learning and optimization for risk-aware supply-chain decisions.

ProcureSight does not directly implement the specialized techniques proposed in these studies, such as blockchain, fuzzy VIKOR, fuzzy MOORA, CRADIS, Least-to-Most Range weighting, or Random Forest-based risk prediction. Instead, the findings from the literature are used as

design considerations for developing a transparent and reviewable procurement decision-support workflow.

The main difference is that ProcureSight begins with transactional purchase-order data and connects several operational analysis stages in one pipeline. The system validates and normalizes purchase-order records, calculates spend and supplier-performance metrics, generates internal P25 price benchmarks, identifies deterministic procurement opportunities, assigns priority scores, and optionally generates grounded advisory narratives.

The comparison between the reviewed literature and ProcureSight is summarized in Table 2.1.

Table 2.1 Comparison of Literature and ProcureSight

Aspect	Literature	ProcureSight
Supplier evaluation	Multi-criteria supplier selection and ranking	Deterministic supplier-performance scoring
Risk assessment	Risk prioritization and predictive risk models	Verified quality, delivery and contract indicators
Criteria weighting	Fuzzy and multi-criteria weighting approaches	Configurable deterministic weights
Price analysis	Supplier selection and optimization contexts	Internal supplier-weighted P25 price benchmarking
Opportunity detection	Generally focused on specific decision objectives	Four deterministic opportunity types

Aspect	Literature	ProcureSight
Decision prioritization	Ranking/optimization methods	Impact, confidence and severity-based priority score
AI explanation	Not the primary focus of the reviewed studies	Optional grounded LLM advisory layer
Data traceability	Depends on the individual method	Supporting metrics and calculation context retained
Final decision	Decision-support oriented	Human review required before procurement action

2.3 Conclusion

The literature survey demonstrates that effective procurement and supply-chain decision support requires transparent evaluation, multiple decision criteria, structured risk assessment and traceable analytical evidence. The reviewed studies provide different approaches for supplier selection, risk prioritization, sustainable supplier evaluation and predictive optimization.

Based on these observations, ProcureSight was designed as an integrated procurement spend analytics and cost-management system. The system combines validated purchase-order data, normalized storage, deterministic analytics, internal price benchmarking, opportunity detection, priority scoring and an optional grounded AI advisory layer.

The literature therefore provides the conceptual foundation for the system, while the implementation focuses on integrating these decision-support principles into a single auditable workflow. ProcureSight does not claim autonomous procurement decision-making or a novel AI/ML algorithm. Instead, it provides reviewable procurement findings that can assist decision-makers while keeping the final decision with human users.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Requirements Analysis

The workflow must accept a fixed CSV structure, validate types and business rules, load dimensions and purchase orders idempotently, compute deterministic analytics, persist benchmarks and opportunities, expose results through HTTP endpoints, and render live data in the frontend. The AI function is optional and must not be required for the analytic pipeline to operate.

Table 3.1. Functional and non-functional requirements

Category	Requirement	Implementation status
Functional	Validate dates, Decimal amounts, and strict Boolean values before import	Implemented
Functional	Create/update normalized dimensions and purchase orders idempotently	Implemented
Functional	Calculate filtered procurement KPIs and opportunity candidates	Implemented
Functional	Generate advisory narrative without exposing raw CSV rows	Implemented, optional
Non-functional	Persist local data in PostgreSQL and manage schema changes	Implemented
Non-functional	Protect server-side API key and return generic 500 errors	Implemented in local scope
Non-functional	Provide role-based access, rate limiting and production observability	Not implemented

3.2 Data Validation

The CSV has 16 fields: po_id, order_date, promised_delivery_date, actual_delivery_date, supplier_id, supplier_name, supplier_country, category, item, business_unit, unit_price, quantity, line_total, payment_terms, on_contract, and quality_rejected. During validation, date fields and Decimal amounts are coerced and Boolean fields use strict accepted representations. Invalid input produces a strict rejection report and prevents a write. The audit reports 47,128 accepted records and zero rejected records for the supplied CSV.

3.3 Feasibility

Technical feasibility is supported by the implemented local Python, FastAPI, PostgreSQL, React, and Docker Compose stack. Operational feasibility beyond local use was not evaluated: there is no user-acceptance evidence, public deployment, load test, or realized-savings workflow. Economic feasibility is not claimed because infrastructure, model-use, and operational cost data are not verified.

3.4 Security Analysis

Implemented controls include Pydantic validation, database constraints, ORM-based queries, validated pagination and filters, CORS allowlisting with credentials disabled, a server-side SecretStr API key configuration, and generic internal-error responses. The current implementation has no authentication, authorization, JWT, RBAC, rate limiting, CSRF controls, audit logging, security headers, or production secret manager.

CHAPTER 4

PROPOSED SYSTEM - PROCURESIGHT

4.1 Architecture

The architecture is batch-oriented. A command-line importer validates the CSV, normalizes dimension data, and upserts purchase orders. PostgreSQL stores dimensions, facts, benchmarks, opportunities, and recommendations. SQL-first services and the optimization engine populate analytics and deterministic findings. FastAPI exposes live endpoints to the React/Vite frontend. The optional LLM is invoked only for a selected opportunity context and can be replaced by a deterministic fallback.

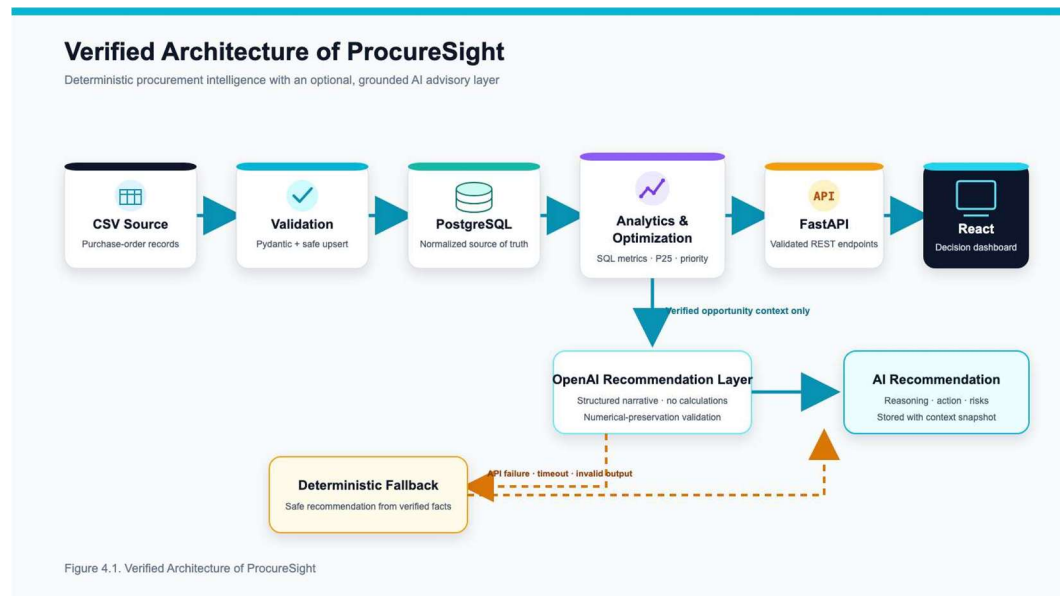


Figure 4.1. Architecture of ProcureSight

4.2 Data Flow

Ingestion is batch/CLI rather than real time. Route functions are synchronous and database sessions are dependency-managed. Successful requests are committed and exceptions are rolled back. Benchmark and

opportunity refresh are explicitly triggered through the optimization engine; no scheduler is present.

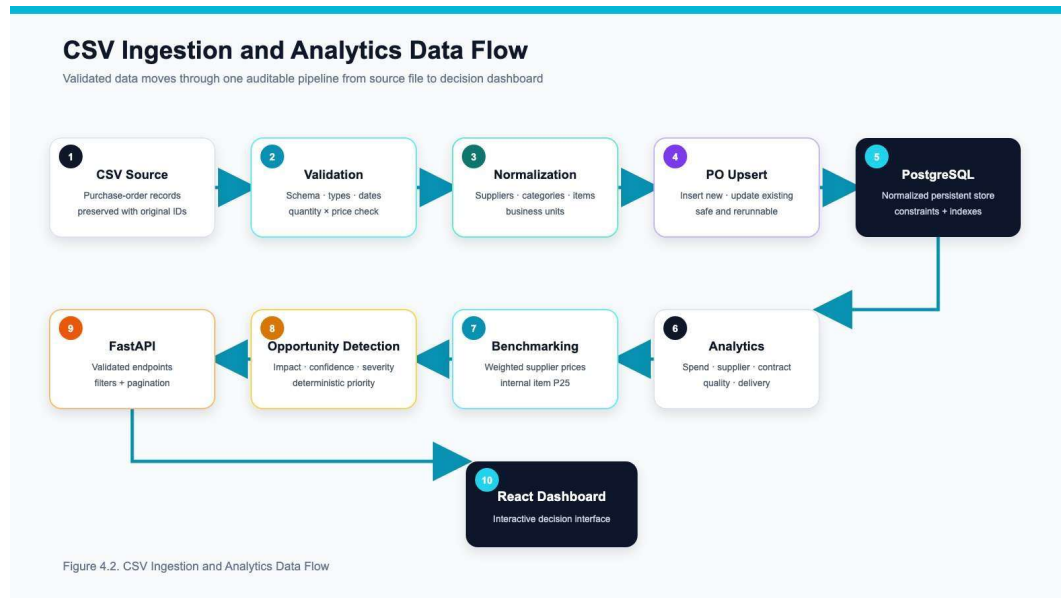


Figure 4.2. CSV ingestion and analytics data flow

4.3 Technology Stack

Table 4.1. Technology stack

Technology	Purpose	Status
Python 3.11+	Backend, scripts, analytics	Implemented
FastAPI	HTTP API and error handlers	Implemented
Pydantic 2	Configuration and schema validation	Implemented
SQLAlchemy 2	ORM and query layer	Implemented
Alembic	Schema migration	Implemented
PostgreSQL 16	Local persistent database through Docker Compose	Implemented
React 19 TypeScript Vite 7	Frontend dashboard	Implemented

Technology	Purpose	Status
Tailwind CSS 4 and Recharts	Styling and charts	Implemented
OpenAI Responses API	Optional advisory narrative	Implemented with fallback
HTTPX	OpenAI client; production code imports it	Packaging issue: declared only as dev dependency

4.4 Database Design

Table 4.2. Database schema

Table	Purpose and key relationships
suppliers	Supplier dimension; supplier_id primary key; unique supplier_name; country
categories	Category dimension; category_id primary key; unique name
items	Item dimension; category_id foreign key; unique item_name/category pair
business_units	Business unit dimension; unique name
purchase_orders	Fact table keyed by po_id; supplier/item/business-unit foreign keys; quantities, prices, dates, flags
item_benchmarks	Per-item P25, median, P75, min/max, supplier coverage and volume/spend
cost_opportunities	Deterministic opportunity type, scores, financial/review amounts, status, JSON metrics
ai_recommendations	Narrative, steps, risk, model/prompt version, context snapshot

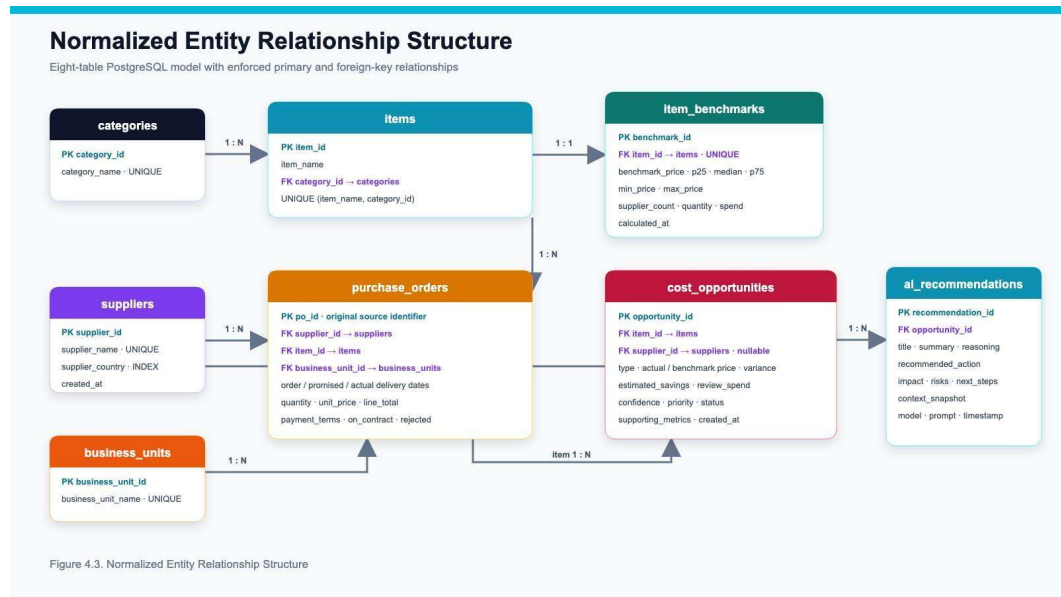


Figure 4.3. Normalized entity relationship structure

4.5 API and User Interface Design

The frontend uses browser fetch through its API client. Implemented pages are Dashboard, Spend Analysis, Cost Optimization, and AI Advisor. The production components use live API calls; mocks are restricted to tests. No upload route exists because ingestion is CLI-only.

CHAPTER 5

METHODOLOGY

5.1 Data Profile

Dataset Provenance: The project uses a structured purchase-order CSV dataset containing transactional procurement records. The provenance and ownership of the dataset could not be independently verified from the project repository. Therefore, no specific company or external organization is attributed as the dataset owner. The dataset is used solely for demonstrating and evaluating the analytical workflow implemented in ProcureSight.

Table 5.1. Dataset profile

Measure	Verified value
Records / columns	47,128 / 16
Date range	2023-01-01 to 2026-06-30
Suppliers / countries	106 / 10
Categories / items / business units	10 / 50 / 5
Total spend	USD 411,183,335.24
Average / median transaction	USD 8,724.8204 / USD 5,802.135
Minimum / maximum transaction	USD 4.52 / USD 68,025.18
Empty values / duplicate PO IDs	0 / 0
On-contract / off-contract orders	40,448 / 6,680
Quality-rejected / late orders	1,029 / 7,700

5.2 Normalization and Analytics

The importer maps suppliers, categories, items, and business units into dimensions and persists purchase orders as facts. Analytics use optional inclusive filters for date, supplier, category, business unit, and country. Implemented measures include total spend, total orders, average order value, spend/share groupings, top-5/top-10 concentration, monthly spend and growth, contract compliance, quality-rejected spend/rate, and late-delivery rate/delay.

5.3 Benchmarking

For each item-supplier group, the system derives a quantity-weighted supplier price. For each item, it computes a continuous P25 across the supplier-level weighted prices. The benchmark stores P25, min, median, P75, max, supplier count, quantity, and spend. It is an internal comparison only: it is not an external-market price, does not perform FX conversion, does not use a transaction-level cheapest-price rule, and has no explicit outlier-removal procedure.

$$\text{weighted price} = \text{SUM}(\text{line total}) / \text{SUM}(\text{quantity}) \quad (5.1)$$

$$\text{price variance percent} = ((\text{actual price} - \text{benchmark price}) / \text{benchmark price}) \times 100 \quad (5.2)$$

$$\text{potential savings} = \max(0, \text{actual price} - \text{benchmark price}) \times \text{quantity} \quad (5.3)$$

For each item i and supplier s , the quantity-weighted supplier price is calculated as:

$$\text{WeightedPrice}(s,i) = \Sigma(\text{UnitPrice}(s,i) \times \text{Quantity}(s,i)) / \Sigma\text{Quantity}(s,i)$$

where the summation is performed over all purchase-order transactions for supplier s and item i .

The internal benchmark price for item i is then calculated as the 25th percentile of the supplier-level weighted prices:

$$\text{Benchmark}(i) = \text{P25}(\{\text{WeightedPrice}(s,i) \mid s \in \text{Suppliers}(i)\})$$

Thus, the benchmark represents the lower-quartile supplier price for the same item based on supplier-level quantity-weighted prices. The benchmark is calculated only from the imported procurement dataset and is therefore an internal comparison rather than an external market-price benchmark.

The system uses deterministic scoring functions to quantify benchmark confidence, supplier performance, and opportunity priority. These scores are calculated by the backend using predefined weights and thresholds and are used only to support review prioritization.

Table 5.2. Formula catalogue

Measure	Exact implemented formula or rule
Percentile position	$(n - 1) \times \text{percentile};$ linearly interpolate between neighboring sorted supplier prices
Confidence	$0.35 \times \text{supplier factor}$ $+ 0.25 \times \text{transaction factor}$ $+ 0.20 \times \text{quantity factor}$ $+ 0.20 \times \text{stability factor}$
Supplier factor	$\min(\text{supplier count} / 5, 1)$

Measure	Exact implemented formula or rule
Transaction factor	$\min(\text{transaction count} / 20, 1)$
Quantity factor	$\min(\log_{10}(\text{quantity} + 1) / 4, 1)$
Stability factor	$\max(0, 1 - \min(P75 - P25 / \text{median}, 1))$
Supplier performance score	$\text{price score} \times 0.40$ $+ \text{quality score} \times 0.25$ $+ \text{delivery score} \times 0.20$ $+ \text{contract score} \times 0.15$
Priority score	$100 \times (\text{impact factor} \times 0.50$ $+ \text{confidence} \times 0.30$ $+ \text{severity} \times 0.20)$

The P25 benchmark is calculated across supplier-level weighted prices for the same item. It is therefore an internal relative benchmark derived from the observed procurement transactions and should not be interpreted as an externally validated market price.

5.4 Opportunity Detection and Ranking

Table 5.3. Opportunity rules and thresholds

Type	Detection threshold	Financial interpretation
Price optimization	At least 3 benchmark suppliers; variance at least 10%; potential savings at least USD 5,000	Estimated potential savings only
Contract leakage	Item-supplier off-contract spend at least USD 100,000	Review spend; estimated savings is zero
Supplier consolidation	At least 3 suppliers; item spend at least USD 250,000; P25-to-P75 dispersion at least 15%	Review candidate; estimated savings is zero
Supplier performance	Supplier spend at least USD 250,000 and score below 75	Review candidate; estimated savings is zero

The opportunity rules are deterministic and configurable; they identify transactions or supplier-item combinations for procurement review rather than representing confirmed savings or mandatory procurement actions.

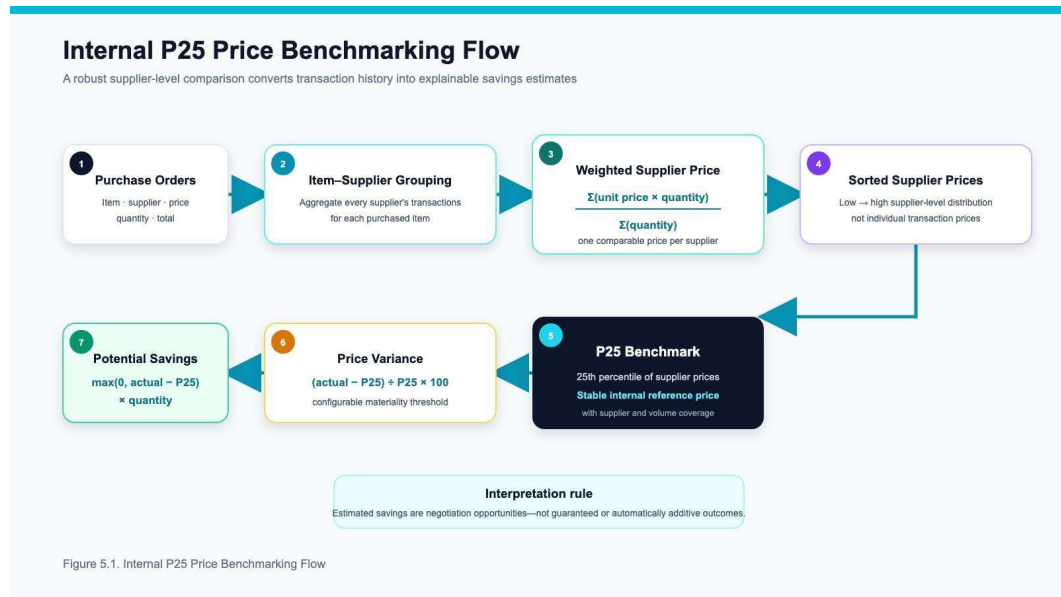


Figure 5.1. Internal P25 price benchmarking flow

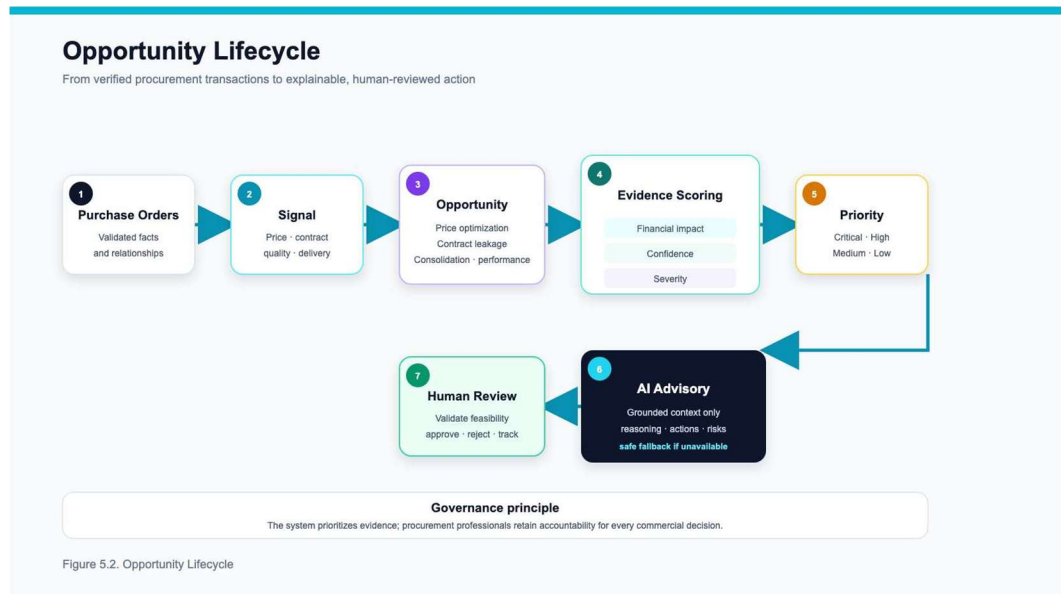


Figure 5.2. Opportunity lifecycle

5.5 Grounded Advisory Method

The advisory component is optional. The OpenAI Responses API uses a configurable model name with gpt-5.4-mini as the default. It receives the selected opportunity ID/type, item, supplier, actual and benchmark price, variance, quantity, savings or review-spend value, confidence, supplier

metrics, and supporting metrics rather than the raw purchase-order dataset. The backend validates structured output through JSON Schema and Pydantic, checks equality of financial impact, and rejects prose numbers absent from context. Two attempts are made; timeout, rate-limit, HTTP, malformed JSON, schema, or numeric-preservation failures store a deterministic fallback. Requests use store: false. Conversation history, retrieval, embeddings, vector databases, and tool calling are not implemented.

CHAPTER 6

IMPLEMENTATION

6.1 Backend and Data Layer

The backend is written in Python and exposes synchronous FastAPI handlers. SQLAlchemy manages ORM queries, while Alembic migrations define the relational schema. The database engine uses `pool_pre_ping=True`. The database session dependency commits successful requests and rolls back on exceptions. Local PostgreSQL 16 is started through Docker Compose with a persistent volume and health check.

6.2 Frontend

The React 19 and TypeScript frontend is built with Vite 7 and styled with Tailwind CSS 4. Recharts presents dashboard and spend-analysis charts. Dashboard provides KPI cards, monthly and categorical charts, and contract composition. Spend Analysis provides date, supplier, category, business-unit, and country filters plus concentration and supplier-ranking displays. Cost Optimization provides summary, type/priority filtering, pagination, and a detail drawer. AI Advisor enables selection of an open opportunity and displays a generated or fallback narrative, risk, and next steps.

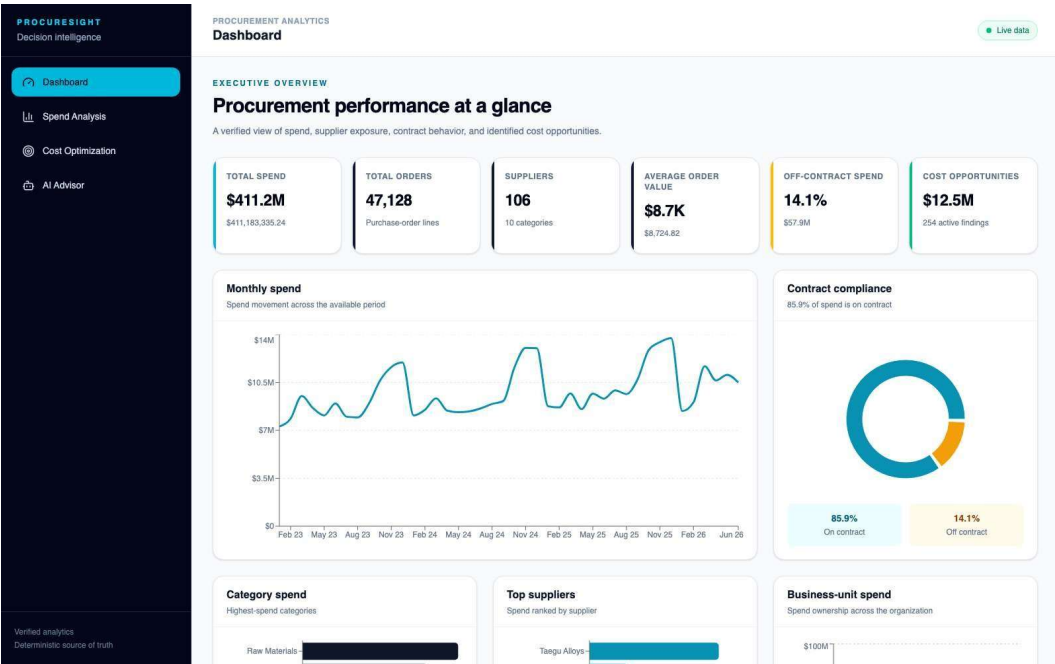


Figure 6.1. Live dashboard screenshot

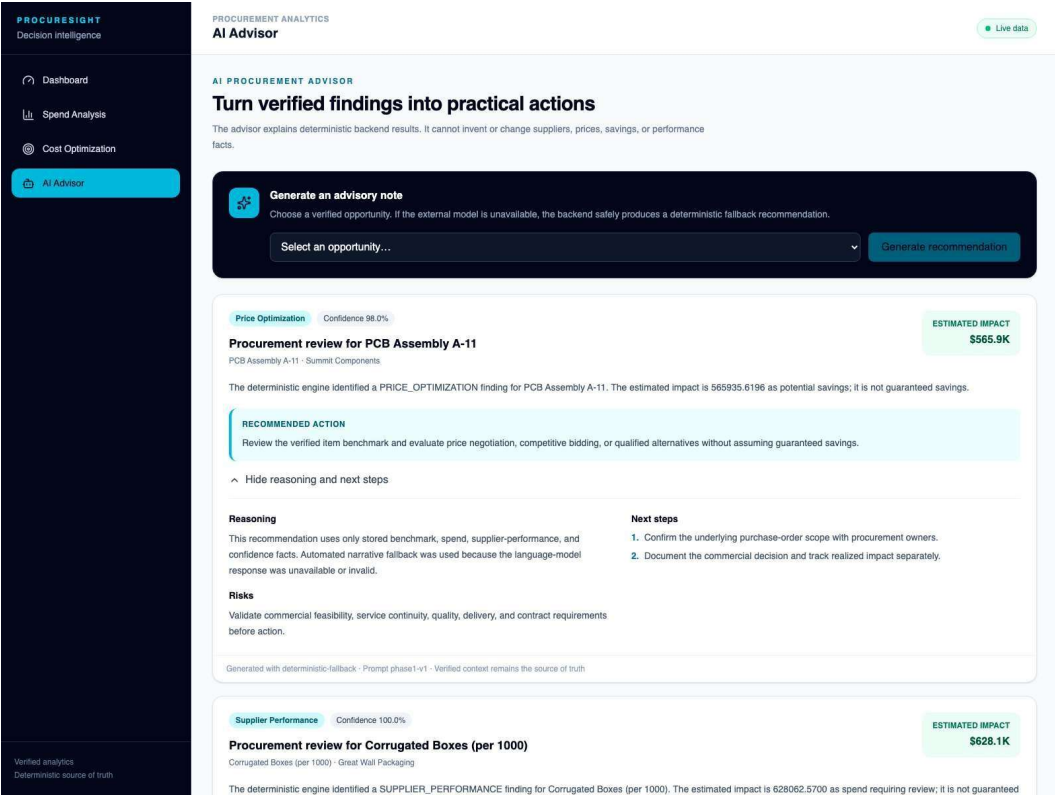


Figure 6.2. AI advisor screenshot

6.3 API Inventory

Table 6.1. API inventory

Method	Endpoint group	Purpose
GET	/health	Health response
GET	/api/dashboard/summary	Dashboard KPIs
GET	/api/analytics/spend/{suppliers,categories,business-units,monthly}	Spend analytics
GET	/api/analytics/supplier-concentration	Top-5/top-10 concentration
GET	/api/analytics/{contract-compliance,quality,delivery}	Operational KPIs
GET	/api/suppliers and /api/suppliers/{id}	Supplier catalog and detail
GET	/api/categories, /api/items, /api/business-units	Reference catalog
GET	/api/benchmarks and /api/benchmarks/{item_id}	Stored internal benchmarks
GET	/api/opportunities, /summary, /{id}	Deterministic findings
POST / GET	/api/recommendations/generate and recommendations resources	Generate/store/retrieve advisory note

6.4 Testing

Table 6.2. Testing summary

Area	Verified result	Interpretation
Backend	51 backend tests passed; one Starlette/httpx deprecation warning	Includes validation, ingestion, analytics, API, optimization, catalog, fallback and numeric safeguards
Frontend	All 6 frontend tests passed	

Area	Verified result	Interpretation
Coverage	Not verified from repository	No coverage percentage should be reported
Performance/load	Not verified from repository	No scalability claim should be reported

6.5 Deployment Status

The verified deployment state is local. Docker Compose provisions the PostgreSQL database, and Uvicorn/Vite are used locally. No Dockerfile, CI workflow, cloud configuration, public deployment URL, Kubernetes configuration, Vercel/Netlify configuration, or production monitoring configuration was found. Production deployment is not verified.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Verified Dataset and Processing Results

Table 7.1. Verified results

Result	Verified value
Validated purchase orders	47,128 accepted; 0 rejected
Suppliers / countries	106 / 10
Categories / items / business units	10 / 50 / 5
Total spend	USD 411,183,335.24
Internal item benchmarks	50
Active deterministic opportunities	254
Backend tests	51 backend tests passed
Frontend tests	6 frontend tests passed

7.2 Opportunity Results

Table 7.2. Opportunity results

Measure	Verified value	Meaning
Price optimization findings	107	Candidate transactions/groups above the internal P25 and configured thresholds

Measure	Verified value	Meaning
Contract leakage findings	144	Material off-contract review candidates
Supplier consolidation findings	1	Review candidate under supplier count, spend, and dispersion thresholds
Supplier performance findings	2	Review candidates below score threshold and above spend threshold
Price-only estimated potential	USD 12,512,042.6726	Potential only; not realized savings
Price-optimization review spend	USD 108,869,726.57	Spend represented by price findings; not savings
Contract-leakage review spend	USD 50,428,294.10	Review spend; not savings

7.3 Discussion

The isolated full-engine execution confirms that the implemented pipeline can deterministically validate the supplied data, store normalized records, derive internal benchmarks, and create prioritized findings. The 254 findings do not establish business value by themselves. They are review candidates, may overlap by type, and need human investigation, commercial validation, and negotiation before any saving can be realized. The price-only potential is derived from internal P25 comparison and should not be interpreted as a market-price gap or a booked financial benefit.

7.4 Limitations of Results

No external benchmark or dataset provenance is verified, no live OpenAI call was verified in the audit and no performance/load evidence exists. The numerical outputs are useful for reproducible internal rule execution, but they do not demonstrate external validity, savings realization, enterprise scale, or AI effectiveness.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

ProcureSight implements a Phase 1 local procurement analytics workflow that transforms a defined CSV into normalized data, deterministic KPIs, internal P25 benchmarks, ranked review opportunities, and optional grounded advisory narratives. The implementation processed 47,128 validated purchase orders and produced 50 internal benchmarks and 254 active findings. Its strongest supported claim is traceability: deterministic calculations generate the financial and priority context, while the optional AI layer is constrained to explain that context. The system does not claim realized savings, an external market benchmark, autonomous procurement decisions, or production readiness.

8.2 Future Work

Table 8.1. Limitations and future work

Current limitation	Reasonable future work
No authentication or authorization	Implement RBAC, secure session design, audit logs, and governance
Manual CSV ingestion and refresh	Add scheduled ingestion and governed refresh workflows
No realized-savings workflow	Add review, assignment, acceptance/rejection and impact tracking
Single currency and internal comparison	Add currency normalization and governed external benchmark sources

Current limitation	Reasonable future work
Local-only operations	Add production deployment, monitoring, backup, rate limiting and observability
No forecasting/anomaly detection	Evaluate data and governance criteria before adding analytical extensions
Dataset provenance unknown	Document owner/source/generator/license and authorization before publication

REFERENCES

- [1] S. Haffar and E. Özceylan, “Blockchain-Based System for Supplier Selection in Sustainable and Leagile Supply Chains,” *IEEE Access*, vol. 12, pp. 139883–139911, 2024, doi: 10.1109/ACCESS.2024.3467059.
- [2] J. K. Runtuk, S. Christanto, and P. K. Ng, “Optimizing Supplier Selection: A Comparative Study of Fuzzy VIKOR and Fuzzy MOORA for Performance-Based Decision Making,” *IEEE Access*, vol. 13, pp. 8456–8468, 2025, doi: 10.1109/ACCESS.2024.3525362.
- [3] K. Qiu, J. Chen, S. Ashraf, and T. Shahid, “Strategic Decision Support System With Probabilistic Linguistic Term Sets: Extended CRADIS Approach for Supply Chain Risk Management in Sports Industry,” *IEEE Access*, vol. 13, pp. 32853–32862, 2024, doi: 10.1109/ACCESS.2024.3416391.
- [4] V. Moretti, E. L. Melo, and J. C. Colmenero, “Least to Most Range (LMR): A Novel Consistent Weighting Multicriteria Method Applied to Green Supplier Selection,” *IEEE Access*, vol. 13, pp. 138986–139000, 2025, doi: 10.1109/ACCESS.2025.3596799.
- [5] S. Shahsavary, M. Alnammi, J. Navaei, and A. Azab, “Risk Prediction in Design of Distributed Supply Chains Using Machine-Learning: A Bi-Objective Optimal Approach,” *IEEE Access*, vol. 14, pp. 77958–77982, 2026, doi: 10.1109/ACCESS.2026.3694980.
- [6] X. Li, V. Culmone, B. De Reyck, and O. S. Yoo, “Automating Procurement Practices Using Artificial Intelligence,” *INFORMS Journal on Applied Analytics*, vol. 55, no. 3, pp. 195–223, 2025, doi: 10.1287/inte.2023.0099.

- [7] J. M. Spreitzenbarth, C. Bode, and H. Stuckenschmidt, “Artificial intelligence and machine learning in purchasing and supply management: A mixed-methods review of the state-of-the-art in literature and practice,” *Journal of Purchasing and Supply Management*, vol. 30, no. 1, Art. no. 100896, 2024, doi: 10.1016/j.pursup.2024.100896.
- [8] M. Guida, F. Caniato, A. Moretto, and S. Ronchi, “The role of artificial intelligence in the procurement process: State of the art and research agenda,” *Journal of Purchasing and Supply Management*, vol. 29, no. 2, Art. no. 100823, 2023, doi: 10.1016/j.pursup.2023.100823.
- [9] A. Westerski, R. Kanagasabai, J. Wong, and H. Chang, “Prediction of Enterprise Purchases Using Markov Models in Procurement Analytics Applications,” *Procedia Computer Science*, vol. 60, pp. 1357–1366, 2015, doi: 10.1016/j.procs.2015.08.209.
- [10] A. Abdulla and G. Baryannis, “A hybrid multi-criteria decision-making and machine learning approach for explainable supplier selection,” *Supply Chain Analytics*, vol. 7, Art. no. 100074, 2024, doi: 10.1016/j.sca.2024.100074.
- [11] H. Shidpour, N. Karimi, G. Baryannis, and M. Shidpour, “A multi-phase analytics framework for supply chain supplier selection and order allocation with delay risks and Industry 4.0 readiness,” *Supply Chain Analytics*, vol. 12, Art. no. 100172, 2025, doi: 10.1016/j.sca.2025.100172.
- [12] “An integrated analytics model for supplier selection and order allocation with machine learning and multi-criteria optimization,” *Decision Analytics Journal*, vol. 16, Art. no. 100599, 2025, doi: 10.1016/j.dajour.2025.100599.

APPENDIX A

VALIDATION AND FORMULA DETAILS

Validation uses a defined 16-column CSV structure, date and Decimal coercion, strict Boolean parsing, and database constraints. Purchase-order quantities must be positive; unit price, line total, opportunity savings, and review spend are nonnegative; delivery dates cannot precede order date; and opportunity type, priority, status, confidence, and priority-score ranges are constrained.

$$\textit{impact factor} = \min(\textit{financial impact} / 1,000,000, 1) \quad (\text{A.1})$$

$$\textit{quality score} = 100 \times (1 - \textit{rejection rate}) \quad (\text{A.2})$$

$$\textit{delivery score} = 100 \times (1 - \textit{late delivery rate}) \quad (\text{A.3})$$

$$\textit{contract score} = \textit{clamp}(100 \times \textit{contract compliance rate}, 0, 100) \quad (\text{A.4})$$

$$\textit{priority score} = 100 \times (\textit{impact factor} \times 0.50 + \textit{confidence} \times 0.30 + \textit{severity} \times 0.20) \quad (\text{A.5})$$

APPENDIX B

VERIFIED DATASET STATISTICS

Measure	Verified value
On-contract spend / off-contract spend	USD 353,329,597.93 / USD 57,853,737.31
On-contract / off-contract rates	85.8258% / 14.1742%
Quality-rejected spend / rate	USD 8,919,669.89 / 2.1834%
Late-order rate	16.3385%
Mean delay among late orders	5.9213 calendar days
Supplier countries	China, Germany, India, Japan, Mexico, Poland, South Korea, UK, USA, Vietnam
Business units	Corporate, Field Services, Manufacturing, R&D, Retail Operations

ProcureSight: An Auditable Procurement Spend Analytics and Opportunity Prioritization System

S Kaviya

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
kaviya.s@rajalakshmi.edu.in

Akshay Venkat Krishna

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701022@rajalakshmi.edu.in

Anirudh S

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701029@rajalakshmi.edu.in

Abstract— ProcureSight is a Phase 1 local procurement decision-support system that transforms validated purchase-order data into spend analytics, internal price benchmarks, deterministic review opportunities, and optional grounded advisory narratives. The implemented workflow accepts a defined CSV structure through command-line ingestion, validates and normalizes the records into PostgreSQL, computes SQL-first analytics, derives supplier-level weighted prices and item-level continuous P25 benchmarks, and detects deterministic price optimization, contract leakage, supplier consolidation, and supplier performance opportunities. The user interface is implemented in React and TypeScript, while the backend uses FastAPI, SQLAlchemy, Alembic, and PostgreSQL running locally through Docker Compose. The advisory layer uses the OpenAI Responses API with a configurable gpt-5.4-mini default and deterministic fallback; financial values are computed in the backend before any narrative is generated. On the supplied 47,128-record dataset, the workflow processed 106 suppliers, 10 countries, 10 categories, 50 items, and 5 business units with total spend of USD 411,183,335.24. The isolated full-engine execution produced 50 internal benchmarks and 254 active opportunities. The USD 12,512,042.6726 result is strictly a price-optimization estimated potential derived from internal comparison rules; it is not a realized saving or an external-market benchmark. Dataset provenance is not verified from the repository, so the dataset is not attributed to a company or external source. The implementation demonstrates reproducible candidate generation and explainable decision support while retaining human review for action and savings realization.

Keywords— *procurement analytics; spend analysis; price benchmarking; cost optimization; supplier performance; contract compliance; deterministic analytics; decision support; FastAPI; PostgreSQL; React; grounded LLM.*

I. INTRODUCTION

Procurement purchase-order data contains information about suppliers, items, quantities, prices, contract status, quality, delivery, and operating units. Converting these transactions into actionable procurement insight requires more than reporting aggregate spend: users must be able to identify material cost or compliance issues, compare prices using a consistent internal reference, prioritize findings, and review the evidence behind each candidate.

ProcureSight implements a traceable workflow from structured purchase-order data to reviewable procurement

candidates. A core design principle is that deterministic services calculate financial and operational context first; the optional language-model layer is restricted to explaining verified context. Consequently, the system does not assign financial calculation, validation, or final procurement decisions to generative output.

The Phase 1 scope is intentionally local and bounded. Data is ingested through a command-line CSV workflow, stored in PostgreSQL, analyzed using SQL/Python services, and exposed through FastAPI to a React/Vite frontend. External market benchmarks, foreign-exchange conversion, forecasting, anomaly detection, scheduled ingestion, authentication/authorization, realized-savings workflows, and production operations are outside the implemented scope.

II. RELATED WORK AND RESEARCH GAP

Recent research on procurement and supply-chain decision support emphasizes supplier evaluation, multi-criteria decision-making, transparency, risk assessment, consistent criteria weighting, and optimization. The following studies were selected in the project review because they provide complementary perspectives on these themes.

A. Haffar and Özceylan—Supplier Selection

Haffar and Özceylan [1] proposed a blockchain-based system for supplier selection in sustainable and agile supply chains. Their system combines multiple supplier-selection criteria with a multi-attribute reverse auction and uses blockchain to improve transparency and traceability. The study demonstrates how technology-supported supplier evaluation can preserve decision evidence. ProcureSight adopts the principle of traceable evidence but uses normalized purchase-order data and deterministic analytical calculations rather than blockchain or reverse auction mechanisms.

B. Runtuk et al.—Multi-Criteria Supplier Ranking

Runtuk et al. [2] compared Fuzzy VIKOR and Fuzzy MOORA for performance-based supplier selection. Their work highlights the role of multiple criteria, criteria weighting, sensitivity, and transparent supplier ranking. ProcureSight similarly evaluates supplier performance using explicit price, quality, delivery, and contract components, but uses deterministic configurable weights

rather than the specific fuzzy MCDM techniques studied in that work.

C. Qiu et al.—Supply-Chain Risk Assessment

Qiu et al. [3] developed a strategic decision-support approach for supply-chain risk management using probabilistic linguistic term sets and an extended CRADIS method. The study addresses uncertain or subjective judgments through structured multi-criteria risk prioritization. ProcureSight does not implement probabilistic or predictive risk modeling in Phase 1; instead, it uses verified operational indicators such as quality rejection, late delivery, and contract compliance.

D. Moretti et al.—Sustainable Supplier Selection

Moretti et al. [4] introduced the Least-to-Most Range (LMR) method for consistent weighting in green supplier selection. The work emphasizes explicit representation of criteria importance and consistency when multiple criteria are considered. ProcureSight uses configurable deterministic weights and thresholds to maintain an interpretable and reproducible scoring process, rather than implementing the LMR method itself.

E. Shahsavary et al.—Machine-Learning-Based Supply-Chain Risk

Shahsavary et al. [5] investigated risk prediction in distributed supply-chain design using machine learning and bi-objective optimization. Their approach combines Random Forest-based risk prediction with optimization to balance cost and risk. The study motivates possible future predictive extensions for ProcureSight, but Phase 1 deliberately remains focused on verified historical procurement metrics and deterministic opportunity detection.

F. Research Gap and Project Contribution

The reviewed studies primarily address specific decision-support objectives such as supplier selection, risk prioritization, sustainable supplier evaluation, or predictive optimization. A practical procurement workflow also needs these analytical activities to be connected to transactional procurement data and operational spend information. This motivates an integrated workflow that can transform purchase-order data into measurable procurement insights, identify potential cost and compliance issues, prioritize findings, and provide interpretable recommendations.

Another common requirement is traceability: rankings, risk indicators, and optimization outputs should be supported by explicit criteria and evidence so that procurement personnel can review and validate them before action. ProcureSight addresses this implementation-level gap by integrating validated purchase-order data, normalized relational storage,

deterministic spend analytics, internal price benchmarking, rule-based opportunity detection, priority scoring, and an optional grounded AI advisory layer in one workflow.

The contribution is therefore integration and traceability rather than a novel AI/ML algorithm. ProcureSight does not directly implement blockchain, Fuzzy VIKOR, Fuzzy MOORA, CRADIS, LMR weighting, or Random Forest risk prediction. These methods are treated as literature-informed design considerations. Final procurement decisions remain under human review.

Table 1. Comparison of reviewed literature and ProcureSight

Aspect	Reviewed literature	ProcureSight
Supplier evaluation	Multi-criteria supplier selection and ranking	Deterministic supplier-performance scoring
Risk assessment	Risk prioritization and predictive risk models	Verified quality, delivery and contract indicators
Criteria weighting	Fuzzy and multi-criteria weighting methods	Configurable deterministic weights
Price analysis	Supplier selection and optimization contexts	Internal supplier-weighted P25 benchmarking
Opportunity detection	Usually specific decision objectives	Four deterministic opportunity types
Prioritization	Ranking/optimization methods	Impact, confidence and severity score
AI explanation	Not the primary focus	Optional grounded LLM advisory
Traceability	Depends on individual method	Supporting metrics and context retained
Final decision	Decision-support oriented	Human review required

III. PROPOSED SYSTEM

ProcureSight follows five principal stages: (1) command-line ingestion of a defined CSV; (2) Pydantic validation and normalized PostgreSQL persistence; (3) SQL-first KPI and supplier-price analysis; (4) deterministic benchmark and opportunity generation; and (5) FastAPI delivery to a React/Vite frontend with an optional advisory layer. PostgreSQL 16 is orchestrated locally with Docker Compose.

analytics and deterministic findings. FastAPI exposes live endpoints to the React/Vite frontend. The optional LLM is invoked only for a selected opportunity context and can be replaced by a deterministic fallback.

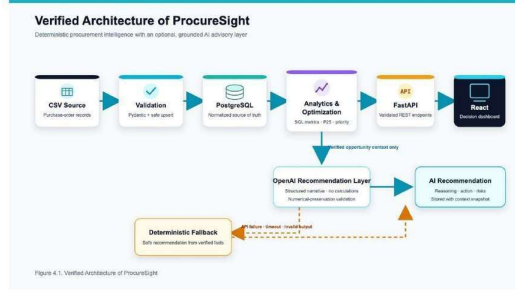


Fig. 1. Verified architecture of ProcureSight.

The architecture is batch-oriented rather than real-time. The importer validates and normalizes dimension data and upserts purchase orders. PostgreSQL stores suppliers, categories, items, business units, purchase orders, benchmarks, opportunities, and recommendations. SQL-first services and the optimization engine calculate analytics and deterministic findings. FastAPI exposes the current stored analytics to the frontend. The optional LLM receives only selected opportunity context and can be replaced by a deterministic fallback.

IV. METHODOLOGY

A. Data Handling and Analytics

The supplied dataset contains 47,128 records and 16 CSV columns: `po_id`, `order_date`, `promised_delivery_date`, `actual_delivery_date`, `supplier_id`, `supplier_name`, `supplier_country`, `category`, `item`, `business_unit`, `unit_price`, `quantity`, `line_total`, `payment_terms`, `on_contract`, and `quality_rejected`. Date fields and Decimal amounts are coerced and Boolean fields use strict accepted representations. Invalid input produces a rejection report and prevents a write. The verified execution accepted all 47,128 records with zero rejected records.

The normalized model separates suppliers, categories, items, and business units into dimensions and stores purchase orders as facts. Analytics support inclusive filtering by date, supplier, category, business unit, and country. Implemented measures include total spend, order count, average order value, spend share, concentration, monthly spend and growth, contract compliance, quality-rejected spend/rate, and late-delivery rate/delay.

B. Internal Price Benchmarking

For each item-supplier group, ProcureSight calculates a quantity-weighted supplier price. For each item, the benchmark is the continuous P25 of the resulting supplier-level weighted prices. The benchmark is an internal comparison only; it is not an external market price, does not use FX conversion, and does not apply a transaction-level cheapest-price rule.

$$WeightedPrice(s,i) = \frac{\sum[UnitPrice \times Quantity]}{\sum[Quantity]} \quad (1)$$

$$Benchmark(i) = ContinuousP25(\{WeightedPrice(s,i) \text{ for suppliers } s \text{ of item } i\}) \quad (2)$$

For a sorted set of n supplier prices, the percentile position is $(n - 1) \times \text{percentile}$, with linear interpolation between neighboring values. The benchmark stores P25, minimum, median, P75, maximum, supplier coverage, quantity, and spend.

C. Confidence and Supplier Performance

The implemented confidence score combines supplier coverage, transaction count, quantity, and price stability. The supplier-performance score combines price, quality, delivery, and contract components. The report's verified formula catalogue defines these calculations as follows.

$$Confidence = 0.35 \times SupplierFactor + 0.25 \times TransactionFactor + 0.20 \times QuantityFactor + 0.20 \times StabilityFactor \quad (3)$$

$$SupplierScore = 0.40 \times PriceScore + 0.25 \times QualityScore + 0.20 \times DeliveryScore + 0.15 \times ContractScore \quad (4)$$

$SupplierFactor = \min(\text{supplier count} / 5, 1)$; $TransactionFactor = \min(\text{transaction count} / 20, 1)$; $QuantityFactor = \min(\log_{10}(\text{quantity} + 1) / 4, 1)$; and $StabilityFactor = \max(0, 1 - \min(|P75 - P25| / \text{median}, 1))$. The underlying quality, delivery, and contract score definitions are based on the validated operational rates described in the project report.

D. Opportunity Detection and Priority Ranking

Table 2. Deterministic opportunity rules.

Opportunity type	Verified rule
Price optimization	At least 3 suppliers; variance $\geq 10\%$; estimated potential $\geq$ USD 5,000
Contract leakage	Item-supplier off-contract spend $\geq$ USD 100,000
Supplier consolidation	At least 3 suppliers; item spend $\geq$ USD 250,000; P25–P75 dispersion $\geq 15\%$
Supplier performance	Supplier spend $\geq$ USD 250,000 and performance score < 75

Opportunity types can overlap. Price-only potential is not added to contract-leakage, consolidation, or supplier-performance review spend. Priority combines financial impact, confidence, and severity. The impact factor is capped at one million US dollars for scoring.

$$Priority = 100 \times (0.50 \times ImpactFactor + 0.30 \times Confidence + 0.20 \times Severity) \quad (5)$$

Priority bands are Critical for scores ≥ 80 , High for ≥ 60 , Medium for ≥ 40 , and Low otherwise. $ImpactFactor = \min(\text{financial impact} / 1,000,000, 1)$.

E. Grounded AI Advisory Layer

The optional advisory layer uses the OpenAI Responses API with a configurable gpt-5.4-mini default. It receives selected opportunity context such as opportunity type, item, supplier, actual and benchmark prices, variance, quantity, potential or review-spend amount, confidence, supplier metrics, and supporting operational metrics. The layer does not receive the full CSV and does not calculate financial values.

Guardrails include a strong system prompt, JSON Schema validation, Pydantic validation, exact-impact equality checking, and rejection of prose numbers that are absent from the verified context. Two attempts are allowed; timeout, rate-limit, HTTP, malformed JSON, schema, or numeric-preservation failures result in a deterministic fallback recommendation. The design therefore treats the LLM as a constrained explanation layer rather than an autonomous procurement engine.

V. IMPLEMENTATION

Table 3. Implemented technology stack.

Layer	Verified technology
Backend	Python 3.11+, FastAPI, Pydantic 2, SQLAlchemy 2, Alembic
Database	PostgreSQL 16 via Docker Compose; psycopg 3
Frontend	React 19, TypeScript, Vite 7, Tailwind CSS 4, Recharts
Testing	Pytest; Vitest and Testing Library
AI	OpenAI Responses API through HTTPX with deterministic fallback

The frontend provides Dashboard, Spend Analysis, Cost Optimization, and AI Advisor pages with live API calls. The backend exposes health, dashboard, analytics, catalog, benchmark, opportunity, and recommendation resources. CSV ingestion remains CLI-only; no file-upload API is implemented. The verified deployment state is local: Docker Compose provisions PostgreSQL and Uvicorn/Vite are used locally. Production deployment, cloud hosting, CI, Kubernetes, public URLs, and production monitoring were not verified.

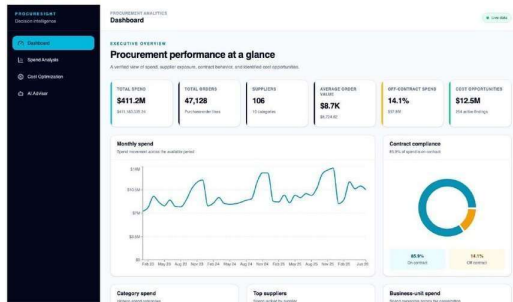


Fig. 2. Live ProcureSight dashboard from the verified project implementation.

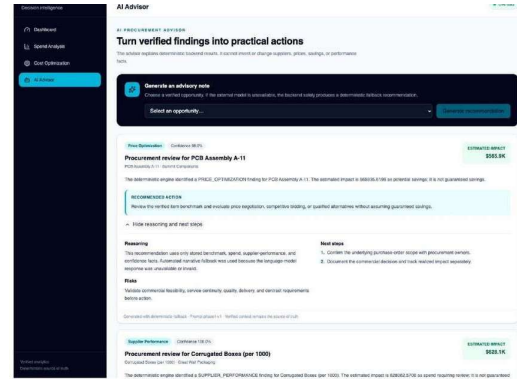


Fig. 3. AI Advisor interface using verified opportunity context.

VI. RESULTS AND DISCUSSION

Table 4. Verified execution results.

Measure	Verified result
Validated purchase orders	47,128 accepted; 0 rejected
Suppliers / countries	106 / 10
Categories / items / business units	10 / 50 / 5
Total spend	USD 411,183,335.24
Internal item benchmarks	50
Active deterministic opportunities	254
Backend tests	51 passed
Frontend tests	6 passed

Table 5. Verified opportunity results.

Opportunity measure	Verified value
Price optimization findings	107
Contract leakage findings	144
Supplier consolidation findings	1
Supplier performance findings	2
Price-only estimated potential	USD 12,512,042.6726
Price-optimization review spend	USD 108,869,726.57
Contract-leakage review spend	USD 50,428,294.10

The isolated full-engine execution confirms that the implemented pipeline can deterministically validate the supplied data, store normalized records, derive internal benchmarks, and create prioritized findings. The 254 findings do not establish business value by themselves. They are review candidates, may overlap by type, and require human investigation, commercial validation, and negotiation before any saving can be realized.

The USD 12,512,042.6726 price-only potential is derived from internal P25 comparison and configured opportunity rules. It must not be interpreted as a market-price gap, booked financial benefit, or realized saving. Similarly, review-spend values represent the spend associated with findings and are not savings.

VII. LIMITATIONS AND FUTURE WORK

Dataset provenance and authenticity are not independently verified from the project repository. The

implementation is single-currency, has no external benchmark source, and has no explicit outlier-removal procedure. Authentication, authorization, RBAC, rate limiting, production observability, backups, and production secret management are outside the current scope. No live OpenAI call was verified in the audit, and no coverage percentage or performance/load evaluation was verified.

Future work includes governed dataset provenance and authorization, role-based access control, scheduled refresh, opportunity review and realized-savings tracking, currency normalization, governed external benchmarks, production deployment and monitoring, forecasting, and anomaly detection. These extensions should be evaluated independently rather than assumed from the current Phase 1 results.

VIII. CONCLUSION

ProcureSight demonstrates a local, auditable procurement decision-support workflow that connects validated purchase-order data with deterministic analytics, internal P25 benchmarks, rule-based candidate generation, priority scoring, and optional grounded narratives. The verified implementation processed 47,128 purchase orders, produced 50 internal benchmarks, and generated 254 active findings across price optimization, contract leakage, supplier consolidation, and supplier performance.

The strongest supported contribution is traceability: procurement findings can be linked to calculated operational and financial context before human review. The project does not support claims of realized savings, external market-price validation, autonomous procurement, predictive risk capability, or production readiness.

REFERENCES

- [1] S. Haffar and E. Özceylan, “Blockchain-Based System for Supplier Selection in Sustainable and Leagile Supply Chains,” *IEEE Access*, vol. 12, pp. 139883–139911, 2024, doi: 10.1109/ACCESS.2024.3467059.
- [2] J. K. Runtuk, S. Christanto, and P. K. Ng, “Optimizing Supplier Selection: A Comparative Study of Fuzzy VIKOR and Fuzzy MOORA for Performance-Based Decision Making,” *IEEE Access*, vol. 13, pp. 8456–8468, 2025, doi: 10.1109/ACCESS.2024.3525362.
- [3] K. Qiu, J. Chen, S. Ashraf, and T. Shahid, “Strategic Decision Support System With Probabilistic Linguistic Term Sets: Extended CRADIS Approach for Supply Chain Risk Management in Sports Industry,” *IEEE Access*, vol. 13, pp. 32853–32862, 2024, doi: 10.1109/ACCESS.2024.3416391.
- [4] V. Moretti, E. L. Melo, and J. C. Colmenero, “Least to Most Range (LMR): A Novel Consistent Weighting Multicriteria Method Applied to Green Supplier Selection,” *IEEE Access*, vol. 13, pp. 138986–139000, 2025, doi: 10.1109/ACCESS.2025.3596799.
- [5] S. Shahsavary, M. Alnammi, J. Navaei, and A. Azab, “Risk Prediction in Design of Distributed Supply Chains Using Machine-Learning: A Bi-Objective Optimal Approach,” *IEEE Access*, vol. 14, pp. 77958–77982, 2026, doi: 10.1109/ACCESS.2026.3694980.
- [6] X. Li, V. Culmone, B. De Reyck, and O. S. Yoo, “Automating Procurement Practices Using Artificial Intelligence,” *INFORMS Journal on Applied Analytics*, vol. 55, no. 3, pp. 195–223, 2025, doi: 10.1287/inte.2023.0099.
- [7] J. M. Spreitzenbarth, C. Bode, and H. Stuckenschmidt, “Artificial intelligence and machine learning in purchasing and supply management: A mixed-methods review of the state-of-the-art in literature and practice,” *Journal of Purchasing and Supply Management*, vol. 30, no. 1, Art. no. 100896, 2024, doi: 10.1016/j.pursup.2024.100896.
- [8] M. Guida, F. Caniato, A. Moretto, and S. Ronchi, “The role of artificial intelligence in the procurement process: State of the art

and research agenda,” *Journal of Purchasing and Supply Management*, vol. 29, no. 2, Art. no. 100823, 2023, doi: 10.1016/j.pursup.2023.100823.

- [9] A. Westerski, R. Kanagasabai, J. Wong, and H. Chang, “Prediction of Enterprise Purchases Using Markov Models in Procurement Analytics Applications,” *Procedia Computer Science*, vol. 60, pp. 1357–1366, 2015, doi: 10.1016/j.procs.2015.08.209.
- [10] A. Abdulla and G. Baryannis, “A hybrid multi-criteria decision-making and machine learning approach for explainable supplier selection,” *Supply Chain Analytics*, vol. 7, Art. no. 100074, 2024, doi: 10.1016/j.sca.2024.100074.
- [11] H. Shidpour, N. Karimi, G. Baryannis, and M. Shidpour, “A multi-phase analytics framework for supply chain supplier selection and order allocation with delay risks and Industry 4.0 readiness,” *Supply Chain Analytics*, vol. 12, Art. no. 100172, 2025, doi: 10.1016/j.sca.2025.100172.
- [12] “An integrated analytics model for supplier selection and order allocation with machine learning and multi-criteria optimization,” *Decision Analytics Journal*, vol. 16, Art. no. 100599, 2025, doi: 10.1016/j.dajour.2025.100599.

ProcureSight Paper.pdf



Osun State University, Osogbo

Document Details

Submission ID

trn:oid::2945:417441638

Submission Date

Oct 6, 2026, 3:44 PM GMT+1

Download Date

Oct 6, 2026, 3:48 PM GMT+1

File Name

ProcureSight Paper.pdf

File Size

403.3 KB

5 Pages

2,657 Words

17,669 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **4 Not Cited or Quoted** 2%
Matches with neither in-text citation nor quotation marks
-  **5 Missing Quotations** 2%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 2%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 4** Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks
- 5** Missing Quotations 2%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 2% Publications
- 3% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

- 1

Internet

ijsrem.com

<1%
- 2

Internet

ieeexplore.ieee.org

<1%
- 3

Submitted works

Liberty University on 2026-05-10

<1%
- 4

Submitted works

AlHussein Technical University on 2026-09-06

<1%
- 5

Internet

huzaifamustafa.com

<1%
- 6

Submitted works

National Institute of Technology, Silchar on 2026-06-17

<1%
- 7

Publication

Vinicius Moretti, Everton L. Melo, João Carlos Colmenero. "Least to Most Range (L...

<1%
- 8

Publication

Shahzaib Ashraf, Tooba Shahid, Jungeun Kim, M. Shazib Hameed, Ibrahim M. Hez...

<1%



AKSHAY VENKAT KRISHNA 230701022 <230701022@rajalakshmi.edu.in>

2027 17th International Conference on Cloud Computing, Data Science & Engineering (Confluence) : Submission (455) has been created.

1 message

Microsoft CMT <noreply@msr-cmt.org>
To: 230701022@rajalakshmi.edu.in

Tue, Oct 6, 2026 at 9:54 PM

Hello,

The following submission has been created.

Track Name: Confluence2027

Paper ID: 455

Paper Title: AI-Driven Procurement Spend Analytics for Cost Management

Abstract:

Procurement spend management is an important aspect of organizational cost control, as large volumes of purchasing data can make it difficult to identify spending patterns, supplier dependencies, and cost-saving opportunities. This paper presents ProcureSight, an AI-driven procurement spend analytics and cost management system designed to transform procurement transaction data into actionable decision support. The proposed system processes procurement data to analyze spending patterns across suppliers, products, categories, and time periods, enabling users to identify major spending areas and potential cost optimization opportunities. An analytics engine performs structured data analysis, while a large language model interprets analytical results and generates contextual recommendations for procurement decision-making. The system integrates these capabilities through an interactive dashboard that provides users with a unified view of procurement performance and spending behavior. By combining structured data analytics with AI-assisted recommendations, ProcureSight reduces the need for manual analysis and improves the accessibility and interpretability of procurement information. The proposed approach demonstrates how artificial intelligence and analytics can support data-driven procurement operations, enhance spending visibility, and assist organizations in identifying opportunities for improved cost management and operational efficiency.

Created on: Tue, 06 Oct 2026 16:24:00 GMT

Last Modified: Tue, 06 Oct 2026 16:24:00 GMT

Authors:

- 230701022@rajalakshmi.edu.in (Primary)
- 230701029@rajalakshmi.edu.in
- kaviya.s@rajalakshmi.edu.in

Secondary Subject Areas: Not Entered

Submission Files:

ProcureSight Paper (1).pdf (638 Kb, Tue, 06 Oct 2026 16:21:45 GMT)

Submission Questions Response: Not Entered

Thanks,
CMT team.

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation
One [Microsoft Way](#)
[Redmond, WA 98052](#)